Bessel functions past the edge of the format

The evaluation paths in `chladni.helmholtz._bessel`

chladni development notes

Abstract

A corner-adapted Fourier–Bessel basis needs $J_\nu(kr)$ at orders ν in the hundreds with kr small, and in that regime `scipy.special.jv` returns exactly zero while the true value is finite. This note develops what the code does instead: a factorisation $J_\nu(x) = S\,e^{\sigma}$ with an *analytic* scale σ, three evaluation branches for S, and the argument that the branches overlap rather than abut. It also derives the error floor — which turns out to be σ itself and not the summation — and the separate per-column scale that leaves the basis matrix well conditioned. Every numerical claim here is measured — the accuracy figures against `mpmath` at 50 digits — and reproducible from the code.

1 The problem

The method of particular solutions expands an eigenfunction near a corner of interior angle ω in the functions $J_\mu(kr)\sin(\mu\theta)$ with $\mu = \ell\nu$ and $\nu = \pi/\omega$. A sharp corner makes ν large, so a basis of a dozen degrees reaches $\mu \sim 300$; meanwhile r is bounded by the domain and k by the eigenvalue sought, so the argument $x = kr$ stays modest. The interesting regime is therefore $\nu \gg x$, where $J_\nu(x)$ is very small.

How small is the difficulty. Table 1 bisects the point at which `scipy.special.jv` first returns a nonzero value. The behaviour is not a gradual loss of accuracy: the routine holds thirteen to fourteen digits and then flushes to exactly 0.0, at $|J| \approx 10^{-291}$ — some seventeen decades above the smallest normal double.

ν	100	200	300	400	500	700
smallest x with $\mathtt{jv}(\nu, x) \neq 0$	0.096	5.32	23.98	51.74	92.06	194.1

Table 1: The underflow edge, bisected against `scipy.special.jv` itself.

A column of zeros is not a small column; it is no column at all. The predecessor code, `lappy`, evaluated exactly this way and its benchmark notes concluded that the sharp corner columns "carry almost no independent information about the domain". Half of that is true — near-dependence at a sharp corner is intrinsic to double precision — and half is an artefact of the representation, which is what this note removes.

2 The factorisation

Write

$$\sigma_J(\nu, x) \;=\; \nu \ln\frac{x}{2} \;-\; \ln\Gamma(\nu+1), \qquad J_\nu(x) \;=\; S\,e^{\sigma_J}, \qquad J'_\nu(x) \;=\; S'\,e^{\sigma_J}. \tag{1}$$

The quantity $e^{\sigma_J} = (x/2)^\nu/\Gamma(\nu+1)$ is the leading term of the ascending series. Two properties make this the right choice of scale rather than one of many.

Proposition 1 ($|S| \le 1$)**.** *For real $x \ge 0$ and $\nu \ge -\frac{1}{2}$, $|J_\nu(x)| \le (x/2)^\nu/\Gamma(\nu+1)$, so $|S| \le 1$ identically.*

Proof. Poisson's integral representation gives

$$J_\nu(x) = \frac{(x/2)^\nu}{\sqrt{\pi}\,\Gamma(\nu+\frac{1}{2})} \int_{-1}^{1} (1-t^2)^{\nu-1/2} \cos(xt)\,\mathrm{d}t,$$

valid for $\nu > -\frac{1}{2}$. Bounding $|\cos(xt)| \le 1$ and using $\int_{-1}^{1}(1-t^2)^{\nu-1/2}\,\mathrm{d}t = \sqrt{\pi}\,\Gamma(\nu+\frac{1}{2})/\Gamma(\nu+1)$ gives the claim. □

So $|S| \le 1$ is a structural invariant, independent of which branch computed it, and the test suite asserts it directly. It also means σ_J is an *upper* scale: the factorisation never has to represent a number larger than one.

The second property is that σ_J is analytic and depends on the evaluation point only through x — it is never a logarithm of the computed value. That matters in the oscillatory regime $\nu \lesssim x$, where $\ln|J_\nu|$ is genuinely singular at every zero. A scale defined as $\ln|J|$ would inherit those poles; this one does not, and the scaled value S carries the oscillation, the sign changes and the zeros itself.

Finally, because σ_J is the same expression in every branch, a caller may subtract two of them — which is exactly what the basis does when it converts from the pointwise scale to a per-column one (§9) — without knowing how either was obtained.

3 Branch A: the ascending series

For x not too large relative to $\sqrt{\nu}$ the defining series is both convergent and accurate. From

$$J_\nu(x) = \sum_{j\ge 0} \frac{(-1)^j (x/2)^{\nu+2j}}{j!\,\Gamma(\nu+j+1)}$$

and (1),

$$S = \sum_{j\ge 0} t_j, \qquad t_0 = 1, \qquad t_j = t_{j-1}\,\frac{q}{j(\nu+j)}, \qquad q = -\frac{x^2}{4}. \tag{2}$$

3.1 The derivative comes free

Each term of the series is $c_j(x/2)^{\nu+2j}$ with c_j independent of x, so

$$\frac{\mathrm{d}}{\mathrm{d}x}\Big[(x/2)^{\nu+2j}\Big] = \frac{\nu+2j}{x}\,(x/2)^{\nu+2j},$$

and therefore

$$x\,S' \;=\; \sum_{j\ge 0} (\nu+2j)\,t_j. \tag{3}$$

The same t_j serve both sums, so one loop with two accumulators yields the value and the derivative. This is worth stating because the alternative — forming J'_ν from $\frac{1}{2}(J_{\nu-1} - J_{\nu+1})$ — would need two more series *and* would produce them with two different scales $\sigma_J(\nu \mp 1, x)$, which then have to be reconciled.

3.2 Accuracy, and the largest term

Equation (2) is an alternating sum. Floating-point summation commits an absolute error of order $\varepsilon_{\mathrm{m}} M$ where $M = \max_j |t_j|$, so the relative error of S is about $\varepsilon_{\mathrm{m}} M/|S|$. The useful question is how big M gets.

For $j \ll \nu$ the recurrence ratio is

$$\left|\frac{t_j}{t_{j-1}}\right| = \frac{x^2/4}{j(\nu+j)} \approx \frac{1}{j}\cdot\frac{x^2}{4\nu},$$

which is the ratio of the exponential series $\sum_j \lambda^j/j!$ with $\lambda = x^2/(4\nu)$. Hence

$$M \lesssim \exp\left(\frac{x^2}{4\nu}\right), \tag{4}$$

attained near $j \approx \lambda$. The estimate is conservative — it ignores both the j-dependence of $\nu + j$ and the $1/\sqrt{2\pi\lambda}$ in Stirling's approximation of the largest term — and measurement confirms it errs on the safe side by a factor of a few. The code uses (4) with $\nu + 1$ in place of ν, which keeps $\nu = 0$ finite and errs further toward the library branch, where the library is exact anyway.

Table 2 shows the measured relative error against `mpmath` at 50 digits, alongside the predicted M. The error tracks $\varepsilon_{\mathrm{m}} M$ closely, and the threshold the code uses is $M \le 10^3$, equivalently $x \lesssim 5.25\sqrt{\nu}$.

ν	x	$x/\sqrt{\nu}$	$M \approx e^{x^2/4\nu}$	relative error
400	20	1.00	1.28	0
400	50	2.50	4.77	1.1×10^{-13}
50	20	2.83	7.39	1.4×10^{-14}
200	50	3.54	2.28×10^{1}	8.5×10^{-14}
400	100	5.00	5.18×10^{2}	1.8×10^{-12}
100	50	5.00	5.18×10^{2}	2.2×10^{-12}
200	100	7.07	2.68×10^{5}	6.8×10^{-7}

Table 2: Branch A against `mpmath` at 50 digits. The error tracks $\varepsilon_{\mathrm{m}} M$. The last row lies past the threshold $M \le 10^3$ and is where the code hands over to branch B.

The loop terminates when $|t_j| \le \varepsilon_{\mathrm{m}}|S|$. Term counts are small: three terms at $x = 0.01$, twenty-four at $(\nu, x) = (400, 50)$, forty-five at $(400, 100)$.

It is worth being precise about the worst case, because it is not where one first looks. Sweeping the whole admitted region $M \le 10^3$, the largest relative error found is 2.9×10^{-11}, at $(\nu, x) = (400, 105)$ — and M there is not exceptional. What is exceptional is $|S| \approx 9.8 \times 10^{-4}$: the point sits near a zero of J_ν, and the error is $\varepsilon_{\mathrm{m}} M/|S|$, with $|S|$ in the *denominator*. Absolute accuracy is untouched; it is relative accuracy that degrades near a zero, as it must. The bound the code reports there is 3.3×10^{-11}, so the diagnostic predicts its own worst case rather than being surprised by it.

3.3 The diagnostic that falls out

M is computed anyway inside the loop, so the bound $\varepsilon_{\mathrm{m}} M/|S|$ costs nothing to report. The code returns it as `rel_err`, and the basis reduces it over the evaluation points to a per-column

relative-accuracy bound. This is the cheap-bound-and-warn pattern the geometry layer uses for `Domain.achievable_prec`: compute a bound in microseconds, warn only when the request exceeds it.

4 Branch B: library evaluation, rescaled

Where M is too large the series is abandoned and `scipy.special.jv` is called directly, then rescaled:

$$S = \operatorname{sgn}\big(J_\nu(x)\big)\exp\big(\ln|J_\nu(x)| - \sigma_J\big).$$

The rescaling is done in logarithms rather than as $J\,e^{-\sigma}$ so that a large σ cannot overflow on the way. A genuine zero of J_ν survives: $\ln 0 = -\infty$ and $e^{-\infty} = 0$.

The derivative uses the symmetric recurrence

$$2J'_\nu(x) = J_{\nu-1}(x) - J_{\nu+1}(x),$$

evaluated with two further `jv` calls. Two remarks on that choice. First, the asymmetric form $J'_\nu = J_{\nu-1} - (\nu/x)J_\nu$ needs only one extra call but suffers cancellation wherever the two terms are close, which the symmetric form does not. Second, `scipy.special.jvp` is not used: it routes every call through a generic n-th derivative formula with a Python loop and a binomial coefficient per term, and profiling a realistic 600×144 basis block put it at 39% of the whole per-k evaluation — more than the Bessel functions it was calling.

Because those two extra calls triple the cost of the library branch, the code takes a flag: when the caller wants values only — which is every Dirichlet boundary, the common case — the derivative is not formed at all. Measured on that same block, 26.3 ms with the derivative against 13.2 ms without.

5 Branch C: the uniform (Debye) asymptotic

Where the library underflows *and* the series is untrustworthy, the code falls back on the classical uniform asymptotic for large order (DLMF 10.19.6–10.19.7). With

$$z = \frac{x}{\nu}, \qquad w = \sqrt{1-z^2}, \qquad p = \frac{1}{w}, \qquad \eta = w + \ln\frac{z}{1+w} \quad (z<1),$$

one has

$$J_\nu(\nu z) \sim \frac{e^{\nu\eta}}{\sqrt{2\pi\nu}\,\sqrt{w}} \sum_{k\ge 0} \frac{U_k(p)}{\nu^k}, \qquad J'_\nu(\nu z) \sim \frac{\sqrt{w}\,e^{\nu\eta}}{\sqrt{2\pi\nu}\,z} \sum_{k\ge 0} \frac{V_k(p)}{\nu^k}, \tag{5}$$

with $U_0 = V_0 = 1$ and

$$U_1 = \tfrac{1}{24}\left(3p - 5p^3\right), \qquad V_1 = \tfrac{1}{24}\left(-9p + 7p^3\right),$$
$$U_2 = \tfrac{1}{1152}\left(81p^2 - 462p^4 + 385p^6\right), \qquad V_2 = \tfrac{1}{1152}\left(-135p^2 + 594p^4 - 455p^6\right),$$
$$U_3 = \tfrac{1}{414720}\left(30375p^3 - 369603p^5 + 765765p^7 - 425425p^9\right),$$
$$V_3 = \tfrac{1}{414720}\left(-42525p^3 + 451737p^5 - 883575p^7 + 475475p^9\right).$$

Reading off the amplitude in (5), the natural Debye scale is

$$\sigma_D = \nu\eta - \tfrac{1}{2}\ln\big(2\pi\nu w\big), \tag{6}$$

and converting to the canonical scale of (1) costs one exponential:

$$S = e^{\sigma_D - \sigma_J} \sum_k \frac{U_k(p)}{\nu^k}, \qquad S' = \frac{w}{z}\, e^{\sigma_D - \sigma_J} \sum_k \frac{V_k(p)}{\nu^k}.$$

Note that the polynomials are sparse in p — only every other power appears — and storing them as dense coefficient tuples invited an off-by-one that showed up as a 2×10^{-7} disagreement with the series. They are stored as explicit (power, coefficient) pairs.

Four terms give at worst 7×10^{-13} over the corner of parameter space where this branch is actually reached ($\nu \gtrsim 520$, $z \lesssim 0.3$; measured against `mpmath` at (ν, x) of $(520, 100)$, $(600, 130)$, $(600, 180)$, $(700, 200)$ and $(1000, 300)$). It degrades only as $z \to 1$, the Airy transition, which is precisely where the library branch is healthy.

6 Branch selection, and why the branches overlap

The rule is:

A	$M \le 10^3$, i.e. $x \lesssim 5.25\sqrt{\nu}$
B	otherwise
C	B returned exactly 0.0

The mask is taken over the whole (points × columns) block, so each branch is a single vectorised call.

Branch C selects *itself*: rather than a threshold in (ν, x), the code runs branch B and falls through wherever the library gave up. That is one fewer tuned constant, and it cannot drift out of date if the library's own underflow edge moves.

The safety of the scheme rests on the branches overlapping rather than abutting. Sweeping $\nu \in [0.2, 600]$ against $x \in [0.5, 100]$ on a 220×220 logarithmic grid and counting cells where `jv` underflows *and* $M > 10^3$:

$$\#\{\text{cells in the gap}\} = 0 \quad \text{out of } 48\,400.$$

Loosening the series threshold to $M > 10^2$ opens the first two gap cells, at $\nu \approx 518$. So the two principal branches are simultaneously valid over a wide strip, and branch C is a safety net for ν beyond roughly 500 rather than a workhorse. The overlap is also the sharpest available test of the implementation: the two branches are compared against each other throughout it, and that comparison fails first if either threshold is edited.

7 The error floor is the scale, not the summation

The bound $\varepsilon_{\mathrm{m}} M/|S|$ of §3 is not the whole story, and the part it misses dominates in the regime this note is about.

The scale $\sigma_J = \nu \ln(x/2) - \ln\Gamma(\nu + 1)$ is assembled from two large numbers. At $(\nu, x) = (400, 50)$ they are 1287.6 and 2000.5. Each is computed to a relative accuracy of about ε_{m}, hence to an *absolute* accuracy of about $\varepsilon_{\mathrm{m}} \times 2 \times 10^3 \approx 2 \times 10^{-13}$ — and an absolute error in a logarithm is a relative error in the value it scales. The realistic bound is therefore

$$\frac{\delta J}{J} \;\approx\; \varepsilon_{\mathrm{m}} \left(\frac{M}{|S|} + \left|\nu \ln(x/2)\right| + \left|\ln\Gamma(\nu + 1)\right| \right). \tag{7}$$

Table 3 shows how badly the first term alone underestimates the truth.

ν	x	$\varepsilon_{\mathrm{m}} M/\lvert S\rvert$	$\varepsilon_{\mathrm{m}}(\lvert\nu \ln \frac{x}{2}\rvert + \lvert \ln \Gamma(\nu+1)\rvert)$	measured
400	50	1.1×10^{-15}	7.3×10^{-13}	1.1×10^{-13}
400	20	2.9×10^{-16}	6.5×10^{-13}	0
100	10	2.9×10^{-16}	1.2×10^{-13}	2.8×10^{-14}

Table 3: The scale, not the summation, sets the floor. Omitting the middle column would understate the error by two orders of magnitude, in the unsafe direction.

Remark 1 (The floor is mostly invisible to the solver). Along a single basis column ν is fixed, so $\ln \Gamma(\nu+1)$ contributes a constant multiplicative error to the whole column. The generalised singular value decomposition that consumes these columns is invariant to column scaling, so it does not see that part at all; what it sees is the summation term and the r-dependence of $\nu \ln(x/2)$. The code reports the honest entrywise number from (7) and says in its docstring that the bound is conservative for a scale-invariant consumer.

The practical consequence is a statement worth making once: *the scaled kernel is a* 5×10^{-13}*-relative object at* $\nu \approx 400$*, not a* 10^{-15} *one.* Tests are written against that number.

8 The second kind

Fundamental solutions use $Y_m(kr)$ at integer order, and the difficulty is the same one with the sign reversed: Y_m blows up as $x \to 0$ where J_ν vanishes. The library overflows rather than underflowing, at the last finite orders measured in Table 4, and again the transition is a cliff at the format's ceiling rather than a decay.

x	10^{-3}	10^{-2}	1	5	20
last finite m	65	80	149	205	294

Table 4: The overflow edge of `scipy.special.yv`; the value there is about 10^{302}.

The standard expansion for integer order $n \geq 1$ is

$$Y_n(x) = -\frac{1}{\pi}\Big(\frac{2}{x}\Big)^n \sum_{j=0}^{n-1} \frac{(n-j-1)!}{j!}\Big(\frac{x^2}{4}\Big)^j + \frac{2}{\pi}\ln\frac{x}{2}\,J_n(x) - \frac{1}{\pi}\Big(\frac{x}{2}\Big)^n \sum_{j\geq 0}\big[\psi(j+1)+\psi(n+j+1)\big]\frac{(-x^2/4)^j}{j!\,(n+j)!}.$$

Only the first sum carries the singular behaviour. Factoring out its $j=0$ term gives the scale and the recurrence

$$\sigma_Y = \ln\Gamma(n) - n\ln\frac{x}{2} - \ln\pi, \qquad a_0 = 1, \quad a_{j+1} = a_j\,\frac{x^2/4}{(n-j-1)(j+1)}, \tag{8}$$

with $S = -\sum_{j=0}^{n-1} a_j$ and, differentiating termwise exactly as in (3), $xS' = -\sum_j (2j-n)\,a_j$. For $n = 0$ the singularity is logarithmic rather than algebraic, so no power-law scale applies and the code sets $\sigma_Y = 0$ and evaluates directly; the order-zero path also uses `y0` and the exact identity $Y_0' = -Y_1$, which is substantially faster than the general-order routines.

The two dropped groups of terms are relatively small by the factor

$$\frac{\text{dropped}}{\text{kept}} \;\sim\; \exp\Big(2n\ln\tfrac{x}{2} - \ln\Gamma(n+1) - \ln\Gamma(n) + \ln\pi\Big), \tag{9}$$

and the branch condition is exactly that this be below 10^{-16}. As with J_ν the two branches overlap: the truncated sum is poor only for $n \lesssim 5$ with $x \gtrsim n$, which is precisely where the library returns an $O(1)$ number with full accuracy.

Remark 2. Choosing the branch on σ_Y rather than on (9) looks equivalent and is not. At $(n, x) = (5, 0.5)$ the scale is already e^9, which sounds safe, while the dropped logarithmic term is 9×10^{-10} of the total. The ratio is the quantity that governs the truncation, and it is the one to test.

9 The per-column scale

The pointwise scale $\sigma_J(\mu, kr)$ of §2 makes each *entry* representable. It does not make a *column* well scaled, because it varies with r along the column. The basis therefore divides each column j by a second, purely analytic quantity $\sigma_j(k)$ that depends on the column and on k but on nothing about the evaluation points:

$$A[:, j] \;=\; \psi_j \, e^{-\sigma_j(k)}, \qquad \psi_j(z) = J_\mu(kr)\sin(\mu\theta).$$

In the evaluation this appears as the single factor $e^{\sigma_J(\mu,kr)-\sigma_j(k)}$, which is where the two scales meet and why both must be canonical analytic numbers.

Point-set independence is what makes this compose with anything the solver does downstream. If the generalised singular value decomposition further rescales columns to unit norm, $\hat{A} = A\,\mathrm{diag}(1/d)$, then the coefficient of ψ_j is $\hat{c}_j \, e^{-\sigma_j}/d_j$: the two log-scales simply add, and neither layer needs to know the other's number. (They are never exponentiated separately. At a $\pi/8$ corner with $kR = 10$ the σ_j span about 1149 nats, so e^{σ_j} ranges over 10^{-499}.)

9.1 Construction

Let R be the largest radius at which the family will be evaluated — the maximum of $|z - \text{origin}|$ over Ω, a fixed geometric constant.[1] Two regimes compete. For $kR < \mu$ the maximum of $|J_\mu|$ over $r \le R$ sits at $r = R$ and is given by the Debye amplitude (6); past $kR = \mu$ the function is oscillatory and the relevant bound is the global maximum over all arguments,

$$\max_{x \ge 0} J_\mu(x) \;\approx\; C_A \, (\mu + \delta)^{-1/3}, \qquad C_A = 0.6748,\ \delta = 0.3,$$

the C_A being the Airy constant and the shift δ fitted so that the same expression is also right at small order (measured ratio to the true maximum in $[0.934, 1.000]$ for $0 \le \mu \le 352$). Writing a for the log of the first and b for the log of the second, the scale is a smooth minimum,

$$\sigma_j \;=\; -\frac{1}{p} \ln\!\left(e^{-pa} + e^{-pb}\right), \qquad p = 4. \tag{10}$$

9.2 Three corrections, each forced by a measurement

The blend must be smooth. A hard min has a discontinuous derivative at the crossover, and $\sigma_j(k)$ sits inside the objective the eigensolver fits a parabola to. The soft minimum costs a factor $2^{1/p} = 1.19$ in the column maximum where the two branches meet, which is the price of that.

[1] Not the corner's clearance, which bounds where the *local expansion* is valid rather than where the column is *evaluated*. Using the clearance is an error of $(r_{\max}/R)^\mu$, which on an L-shape with twelve degrees per corner was measured at 5.7×10^{10}.

The amplitude must be floored. The Debye amplitude $-\frac{1}{2}\ln(2\pi\mu w)$ diverges as $w \to 0$, which is exactly the crossover. Measured as a second difference of σ_j in k across $kR = \mu$ at $\mu = 20$:

unfloored amplitude, $p = 1$	8.4×10^5
unfloored amplitude, hard min	1.9×10^3
floored amplitude, $p = 4$ (as shipped)	6.5

The middle row is the instructive one: *unfloored, the smooth blend is worse than* min, because min clips the divergence while $\ln(e^{-a} + e^{-b})$ follows it in. The floor is the load-bearing correction; the soft blend removes the kink that remains.

The floor is placed at the Airy transition width, where the asymptotic ceases to be valid anyway, and its constant is *solved* rather than tuned: requiring the floored amplitude to equal the global maximum exactly at $z = 1$ (where $\eta = 0$) gives

$$-\tfrac{1}{2}\ln\big(2\pi\mu\, w_{\text{floor}}\big) = \ln\big(C_A(\mu+\delta)^{-1/3}\big) \quad\Longrightarrow\quad w_{\text{floor}} = \frac{C_A^{-2}}{2\pi}\cdot\frac{(\mu+\delta)^{2/3}}{\mu},$$

with $C_A^{-2}/(2\pi) = 0.349518$. So the envelope and the cap *meet* at the crossover rather than merely crossing.

The positive part must be smooth too. Writing $q = 1 - z^2$, the floored amplitude uses $\big(q_+^2 + w_{\text{floor}}^4\big)^{1/4}$. Past the crossover q turns negative, and using q^2 unmodified lets the amplitude grow again, driving the column maximum to 27.7. Clamping with $\max(q, 0)$ fixes the magnitude but leaves a jump in the second derivative; the code uses the C^∞ form

$$q_+ = \tfrac{1}{2}\Big(q + \sqrt{q^2 + w_{\text{floor}}^4}\Big),$$

which has the same two limits.

9.3 What it achieves

Over μ from 0.05 to 320 and kR from 0.5 to 300, the maximum of each scaled column lies in [0.802, 1.185] — a spread of 1.48 — against raw columns spanning hundreds of orders of magnitude on the same grid. The second difference of σ_j in k is at most 20.8 at $\mu = 2$, 3.1 at $\mu = 20$ and 0.5 at $\mu = 200$.

Finally, σ_j depends on k and R only through the product kR. Under the geometric normalisation $\Omega \mapsto s\Omega$, for which $\lambda \mapsto s^{-2}\lambda$ and hence $k \mapsto k/s$, the product is unchanged and every column scale is invariant *exactly*, bit for bit. That is asserted as an equality rather than to a tolerance: if it ever needs a tolerance, the formula has grown a term that should not be there.

10 Summary

	what it computes	accuracy
A, series	S and S' from one loop	$\varepsilon_{\text{m}} M/\lvert S\rvert$; $\le 3 \times 10^{-11}$ over $M \le 10^3$
B, library	`jv`, rescaled in logs	$\le 2 \times 10^{-13}$ up to $\nu \approx 400$
C, Debye	four-term uniform asymptotic	$\le 7 \times 10^{-13}$ where reachable
floor	$\varepsilon_{\text{m}}\,\lvert\sigma_J\rvert$, from §7	$\sim 10^{-13}$ at $\nu = 400$

Away from the zeros of J_ν — which is almost everywhere — the series delivers 10^{-13}. The 3×10^{-11} is the price of asking for *relative* accuracy at a point where the function is a thousand times smaller than its own series terms, and the reported bound says so in advance.

The three branches overlap with no gap over the parameter range the solver uses; the factorisation makes $|S| \leq 1$ a checkable invariant; the derivative shares the value's loop and its scale; and a second, purely analytic per-column scale turns a 500-decade range into a 1.5-fold one without ever consulting the evaluation points.

References

- NIST Digital Library of Mathematical Functions, §10.2 (series), §10.6 (recurrences), §10.19 (uniform asymptotics for large order). https://dlmf.nist.gov/
- G. N. Watson, *A Treatise on the Theory of Bessel Functions*, 2nd ed., §3.31 for the bound of Proposition 1.
- F. W. J. Olver, *Asymptotics and Special Functions*, ch. 12, for the U_k and V_k polynomials and the range of validity of (5).